ESPE UNIVERSIDAD DE LAS FUERZAS ARMADAS INNOVACIÓN PARA LA EXCELENCIA	INFORME DE PRÁCTICA DE LABORATORIO

DEPARTAMENTO:	Ciencias de la Computacion	**CARRERA:**	Ingenieria en Tecnologias de la Informacion		
ASIGNATURA:	Sistemas Operativos	**NIVEL:**	3	**FECHA:**	09/06/2026
DOCENTE:	Christian Alfredo Coronel Guerrero	**PRÁCTICA N°:**	1	**CALIFICACIÓN:**	

Análisis Experimental de Multiprocesamiento, Hilos, Concurrencia e Interbloqueo en Entornos Linux CentOS mediante Python 3

Jeffry David Vera Borja , Erick Saint Cotera Becerra

RESUMEN

El presente informe documenta los resultados obtenidos en la Práctica de Laboratorio N°2 de la asignatura Sistemas Operativos, ejecutada sobre una máquina virtual CentOS 7 desplegada en Oracle VirtualBox. El objetivo central consistió en evaluar experimentalmente las diferencias de rendimiento, gestión de memoria y sincronización entre los módulos threading y multiprocessing de Python 3.6.8. Se implementaron cuatro scripts orientados a la creación de procesos hijos identificados por PID, instanciación de hilos nominales, simulación de concurrencia mediante salidas asíncronas y provocación controlada de interbloqueo mediante locks cruzados. En la etapa de benchmarking se ejecutó un contador iterativo de 1.000.000.000 ciclos primero sobre un único proceso (25.0464 s) y luego distribuido en dos procesos simultáneos de 500.000.000 iteraciones cada uno (13.7032 s), obteniendo un speedup de aproximadamente 1.83×. Los resultados confirman que el módulo multiprocessing elude el Global Interpreter Lock (GIL) de CPython al crear instancias independientes del intérprete, distribuyendo la carga de cómputo en núcleos físicos distintos del procesador. Se verificó además que los interbloqueos representan un riesgo crítico en sistemas de producción que requieren estrategias explícitas de sincronización.

Palabras Claves: Multiprocesamiento, Hilos, Concurrencia.

1. INTRODUCCIÓN

La gestión eficiente de procesos e hilos constituye uno de los pilares fundamentales en el diseño y operación de sistemas operativos modernos. A medida que la Ley de Moore comenzó a evidenciar sus límites físicos en cuanto a la escalabilidad de frecuencias de reloj individuales, la industria del hardware migró hacia arquitecturas multinúcleo (multi-core) como estrategia dominante para incrementar el rendimiento computacional. Este paradigma impone a los sistemas operativos y a los desarrolladores de software la responsabilidad de explotar el paralelismo a nivel de procesos e hilos de forma correcta y eficiente.

En el contexto de la programación en Python, la existencia del Global Interpreter Lock (GIL) en la implementación de referencia CPython introduce restricciones sobre el paralelismo verdadero a nivel de hilos, haciendo del módulo multiprocessing la alternativa canónica para la distribución de carga computacional intensiva en múltiples núcleos. El entorno Linux, específicamente CentOS 7, ofrece un planificador de procesos basado en el algoritmo Completely Fair Scheduler (CFS) que gestiona de manera óptima la asignación de tiempo de CPU a los procesos concurrentes.

La presente práctica tiene como propósito llevar a cabo un análisis empírico y comparativo de los mecanismos de concurrencia disponibles en Python 3 bajo un entorno virtualizado Linux, validando

mediante mediciones reales de tiempo los fundamentos teóricos sobre procesos, hilos, sincronización e interbloqueo.

2. OBJETIVOS

- Implementar y configurar un entorno virtualizado bajo CentOS 7 Linux para la ejecución de scripts en Python 3 enfocados en concurrencia y paralelismo.
- Evaluar experimentalmente las diferencias de rendimiento, gestión de memoria y sincronización entre hilos (threading) y procesos (multiprocessing) mediante benchmarks cronometrados.
- Analizar el comportamiento de condiciones de carrera e interbloqueos (deadlocks) en la ejecución de hilos concurrentes, identificando sus causas estructurales y su impacto en sistemas de producción.

3. MARCO TEÓRICO

3.1 Procesos vs. Hilos en Linux

En el kernel de Linux, un proceso es la unidad básica de ejecución con su propio espacio de direcciones virtuales, tabla de descriptores de archivo y contexto de hardware. Cada proceso se representa mediante un Process Control Block (PCB) —denominado task_struct en Linux— que almacena el estado (Running, Sleeping, Stopped, Zombie), el PID, el PPID y los punteros a las estructuras de memoria virtual. El aislamiento garantizado por la MMU impide que un proceso acceda al espacio de otro, brindando protección al sistema.

Un hilo es una unidad de ejecución ligera que comparte el espacio de direcciones del proceso padre, incluyendo el heap y las variables globales. Linux los implementa como tareas del kernel con la misma estructura task_struct, diferenciándose en que comparten recursos mediante los flags CLONE_VM y CLONE_FILES en la llamada clone(). Esta arquitectura reduce el coste de creación y cambio de contexto, pero introduce la necesidad de sincronización explícita para evitar condiciones de carrera.

3.2 Global Interpreter Lock (GIL) de Python

El GIL es un mutex interno de CPython que garantiza que únicamente un hilo pueda ejecutar bytecode Python en un instante dado, independientemente de los núcleos disponibles. Simplifica la gestión de referencias del recolector de basura, pero serializa el acceso al intérprete en aplicaciones CPU-bound. El módulo multiprocessing resuelve esto creando subprocesos independientes mediante fork(), cada uno con su propio GIL, permitiendo ejecución paralela real en distintos núcleos físicos.

3.3 Concurrencia, Sincronización y Condiciones de Carrera

La concurrencia es la propiedad de un sistema donde múltiples tareas progresan en intervalos solapados. Una condición de carrera ocurre cuando el resultado depende del orden relativo e impredecible de los accesos a recursos compartidos entre hilos. El objeto Lock del módulo threading implementa exclusión mutua: acquire() bloquea el lock y release() lo libera. El uso incorrecto puede derivar en interbloqueos.

3.4 Interbloqueo (Deadlock) — Condiciones de Coffman

Un interbloqueo es una situación donde dos o más hilos quedan bloqueados indefinidamente, cada uno esperando un recurso retenido por el otro. Las cuatro condiciones necesarias y suficientes (Coffman et al., 1971) son:

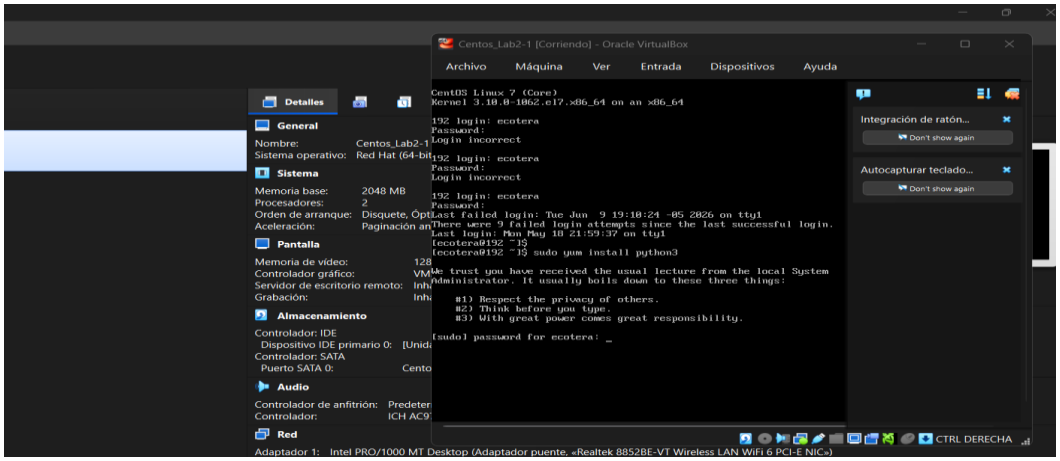
- Exclusión mutua: el recurso no puede ser usado simultáneamente por más de un proceso.

- Retención y espera: un proceso retiene al menos un recurso mientras espera adquirir recursos adicionales.
- No desalojo: los recursos no pueden ser arrebatados forzosamente; solo el proceso que los retiene puede liberarlos.
- Espera circular: existe una cadena circular donde cada proceso espera un recurso retenido por el siguiente.

4. DESCRIPCIÓN DEL PROCEDIMIENTO

4.1 Despliegue del entorno virtualizado CentOS 7

Se configuró una máquina virtual mediante Oracle VirtualBox utilizando una imagen ISO de CentOS 7 (x86_64). Los parámetros incluyeron 2 vCPUs mapeados a núcleos físicos del procesador anfitrión, 2 GB de RAM y 20 GB de disco en formato VDI. El sistema se instaló con perfil Minimal Install para reducir el consumo de recursos.



4.2 Instalación de Python 3.6.8 mediante YUM

Se actualizaron los repositorios del sistema y se instaló Python 3.6.8 mediante el gestor de paquetes YUM. La versión detectada con python3 --version confirmó la instalación exitosa del intérprete CPython con todos los módulos de biblioteca estándar necesarios (threading, multiprocessing, os, time).

```
---> Paquete python3.x86_64 0:3.6.8-21.el7_9 debe ser instalado
--> Procesando dependencias: python3-libs(x86-64) = 3.6.8-21.el7_9 para el paquete: python3-3.6.8-21
.el7_9.x86_64
--> Procesando dependencias: python3-setuptools para el paquete: python3-3.6.8-21.el7_9.x86_64
--> Procesando dependencias: python3-pip para el paquete: python3-3.6.8-21.el7_9.x86_64
--> Procesando dependencias: libpython3.6m.so.1.0()(64bit) para el paquete: python3-3.6.8-21.el7_9.x
86_64
--> Ejecutando prueba de transacción
---> Paquete python3-libs.x86_64 0:3.6.8-21.el7_9 debe ser instalado
--> Procesando dependencias: libtirpc.so.1()(64bit) para el paquete: python3-libs-3.6.8-21.el7_9.x86
_64
---> Paquete python3-pip.noarch 0:9.0.3-8.el7 debe ser instalado
---> Paquete python3-setuptools.noarch 0:39.2.0-10.el7 debe ser instalado
--> Ejecutando prueba de transacción
---> Paquete libtirpc.x86_64 0:0.2.4-0.16.el7 debe ser instalado
--> Resolución de dependencias finalizada

Dependencias resueltas

================================================================================================
 Package                  Arquitectura       Versión                  Repositorio         Tamaño
================================================================================================
Instalando:
 python3                  x86_64             3.6.8-21.el7_9           updates             71 k
Instalando para las dependencias:
 libtirpc                 x86_64             0.2.4-0.16.el7           base                89 k
 python3-libs             x86_64             3.6.8-21.el7_9           updates            7.0 M
 python3-pip              noarch             9.0.3-8.el7              base               1.6 M
 python3-setuptools       noarch             39.2.0-10.el7            base               629 k

Resumen de la transacción
================================================================================================
Instalar  1 Paquete (+4 Paquetes dependientes)

Tamaño total de la descarga: 9.3 M
Tamaño instalado: 48 M
Is this ok [y/d/N]:
```

Se creó el directorio ~/lab_procesos mediante mkdir ~/lab_procesos && cd ~/lab_procesos. Todos los scripts de la práctica fueron codificados y almacenados en esta ubicación.

```
Tamaño total de la descarga: 9.3 M
Tamaño instalado: 48 M
Is this ok [y/d/N]: y
Downloading packages:
(1/5): python3-3.6.8-21.el7_9.x86_64.rpm                    |  71 kB  00:00:00
(2/5): libtirpc-0.2.4-0.16.el7.x86_64.rpm                   |  89 kB  00:00:00
(3/5): python3-setuptools-39.2.0-10.el7.noarch.rpm          | 629 kB  00:00:00
(4/5): python3-pip-9.0.3-8.el7.noarch.rpm                   | 1.6 MB  00:00:01
(5/5): python3-libs-3.6.8-21.el7_9.x86_64.rpm               | 7.0 MB  00:00:03
--------------------------------------------------------------------------------
Total                                              2.8 MB/s | 9.3 MB  00:00:03
Running transaction check
Running transaction test
Transaction test succeeded
Running transaction
  Instalando    : libtirpc-0.2.4-0.16.el7.x86_64                              1/5
  Instalando    : python3-setuptools-39.2.0-10.el7.noarch                     2/5
  Instalando    : python3-pip-9.0.3-8.el7.noarch                              3/5
  Instalando    : python3-3.6.8-21.el7_9.x86_64                               4/5
  Instalando    : python3-libs-3.6.8-21.el7_9.x86_64                          5/5
  Comprobando   : libtirpc-0.2.4-0.16.el7.x86_64                              1/5
  Comprobando   : python3-libs-3.6.8-21.el7_9.x86_64                          2/5
  Comprobando   : python3-3.6.8-21.el7_9.x86_64                               3/5
  Comprobando   : python3-setuptools-39.2.0-10.el7.noarch                     4/5
  Comprobando   : python3-pip-9.0.3-8.el7.noarch                              5/5

Instalado:
  python3.x86_64 0:3.6.8-21.el7_9

Dependencia(s) instalada(s):
  libtirpc.x86_64 0:0.2.4-0.16.el7          python3-libs.x86_64 0:3.6.8-21.el7_9
  python3-pip.noarch 0:9.0.3-8.el7          python3-setuptools.noarch 0:39.2.0-10.el7

¡Listo!
[root@192 ~]#
```

4.4 Script proceso.py — Creación de proceso hijo

Se codificó el script proceso.py que instancia un proceso hijo mediante multiprocessing.Process(target=...) y retorna su PID mediante os.getpid(). Al ejecutarse retornó el PID 1602, valor asignado por la tabla de procesos del kernel CentOS. Esto confirma que el módulo multiprocessing invoca correctamente la llamada al sistema fork(), creando un proceso hijo independiente del proceso padre.

```
from multiprocessing import Process
import os

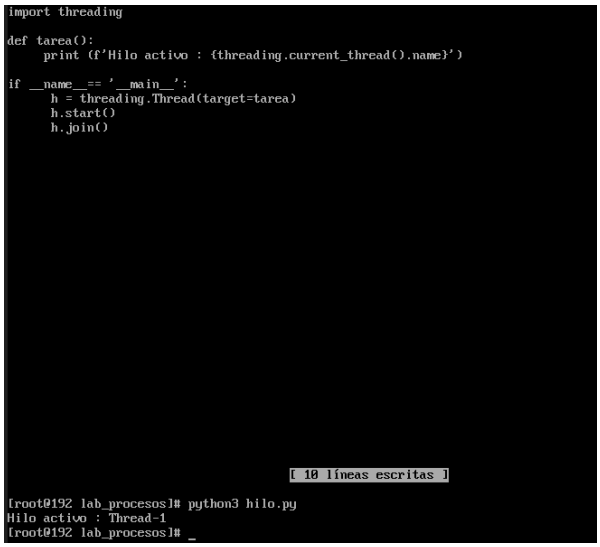
def tarea():
    print(f'Proceso hijo:{os.getpid()}')
if __name__== '__main__':
    p = Process (target=tarea)
    p.start()
    p.join()

[ 10 líneas escritas ]

[root@192 lab_procesos]# python3 proceso.py
Proceso hijo:1602
[root@192 lab_procesos]# _
```

4.5 Script hilo.py — Instanciación de hilo

El script hilo.py instanció un hilo mediante threading.Thread() e imprimió su identificador lógico como 'Thread-1', nombre asignado secuencialmente por el módulo threading en orden de creación. A diferencia de los procesos, el hilo comparte el espacio de memoria y el PID del proceso padre. El resultado confirma la correcta instanciación del hilo dentro del proceso principal.



4.6 Script concurrencia.py — Ejecución asíncrona de hilos

Los hilos A y B ejecutaron sus bucles de impresión con retardos time.sleep(), generando una salida intercalada no determinista entre sus iteraciones. Este comportamiento evidencia la naturaleza concurrente del scheduler de hilos del kernel, que alterna la ejecución entre los hilos en función del quantum de tiempo disponible. La secuencia observada no es reproducible entre ejecuciones, confirmando el carácter probabilístico de la planificación concurrente.



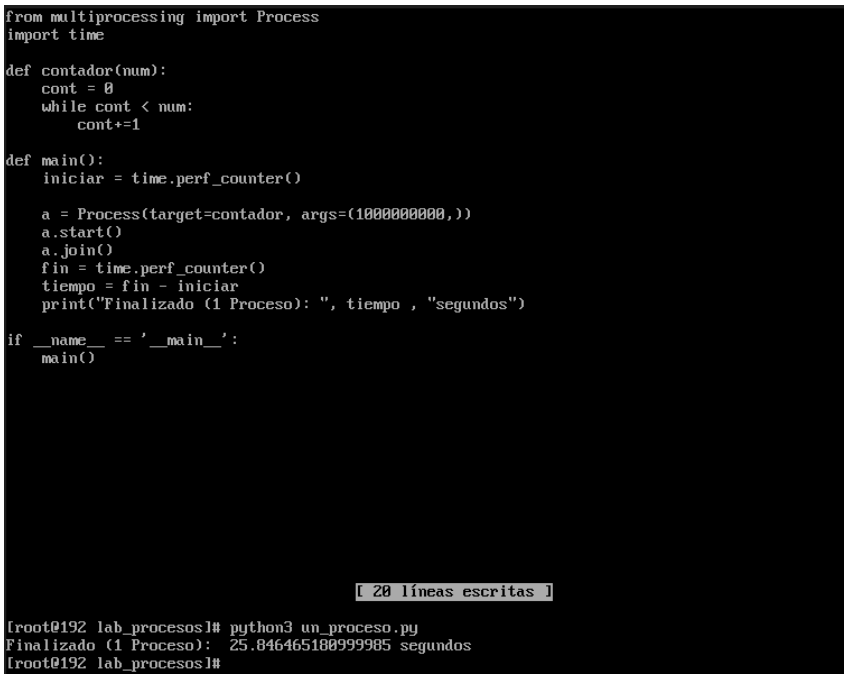
4.7 Script interbloqueo.py — Deadlock provocado

El programa se detuvo indefinidamente al ejecutar los hilos con locks cruzados. El Hilo-1 adquirió lock1 y quedó en espera de lock2, mientras el Hilo-2 adquirió lock2 y esperó lock1. Se satisfacen las cuatro condiciones de Coffman: exclusión mutua, retención y espera, no desalojo y espera circular (ciclo: Hilo1 → lock2 → Hilo2 → lock1 → Hilo1). El programa requirió interrupción manual mediante Ctrl+C.



4.8 Parte 2 — Benchmark de rendimiento: 1 proceso vs. 2 procesos

Se implementó una función de cómputo intensivo con un bucle de 1.000.000.000 iteraciones. Primero se ejecutó en un único proceso midiendo el tiempo con time.time() (resultado: 25.0464 s). Luego se dividió en dos procesos simultáneos de 500.000.000 iteraciones cada uno, iniciados con Process.start() y sincronizados con Process.join() (resultado: 13.7032 s). El speedup obtenido fue de aproximadamente 1.83×.



```
def contador(num):
    cont = 0
    while cont < num:
        cont += 1

def main():
    iniciar = time.perf_counter()

    a = Process(target=contador, args=(500000000,))
    b = Process(target=contador, args=(500000000,))

    a.start()
    b.start()

    a.join()
    b.join()

    fin = time.perf_counter()
    tiempo = fin - iniciar
    print("Finalizo (2 Procesos): " , tiempo, "segundos")

if __name__ == '__main__':
    main()

                                [ 27 líneas escritas ]

[root@192 lab_procesos]# python3 dos_proceso.py
python3: can't open file 'dos_proceso.py': [Errno 2] No such file or directory
[root@192 lab_procesos]# python3 dos_procesos.py
Finalizo (2 Procesos):  13.78323715299939 segundos
```

5. ANÁLISIS DE RESULTADOS

5.1 Tabla de rendimiento: 1 proceso vs. 2 procesos

La siguiente tabla resume los tiempos medidos y el speedup calculado para ambas configuraciones de la Parte 2:

Configuración	Iteraciones	Tiempo (s)	Speedup
1 proceso	1,000,000,000	**25.0464 s**	1.00× (base)
2 procesos paralelos	500,000,000 c/u	**13.7032 s**	**≈ 1.83×**

La reducción de 25.0464 s a 13.7032 s representa una mejora de rendimiento del 45.3%, equivalente a un speedup de 1.83×. Este resultado valida que multiprocessing distribuye carga de cómputo en múltiples núcleos físicos, superando la barrera del GIL.

5.2 Tabla comparativa: Procesos vs. Hilos

La siguiente tabla resume las diferencias estructurales entre los dos mecanismos de concurrencia evaluados:

Aspecto	Proceso (multiprocessing)	Hilo (threading)	Implicación práctica
Memoria	Espacio aislado (PCB independiente)	Espacio compartido (heap y variables globales)	—

Paralelismo real	Sí — distribuye en núcleos físicos	Limitado por GIL de CPython	—
Comunicación	Queue, Pipe (IPC explícito)	Variables compartidas + Lock	—
Seguridad ante fallos	Alta — fallo aislado por proceso	Baja — hilo puede corromper estado global	—
Coste de creación	Alto (fork + nuevo intérprete)	Bajo (thread en mismo intérprete)	—
Sincronización	No requerida entre procesos	Obligatoria (Lock, Semaphore, Condition)	—

Nota: La tabla fundamenta los resultados experimentales obtenidos en los scripts de la Parte 1 y el benchmark de la Parte 2.

6. DISCUSIÓN

Los resultados experimentales validan de forma contundente el modelo teórico del Global Interpreter Lock (GIL) de CPython. En la configuración de un único proceso, el intérprete Python ejecuta el bucle de 10^9 iteraciones de manera estrictamente secuencial: aunque CentOS 7 disponga de múltiples núcleos físicos, el GIL serializa la ejecución del bytecode impidiendo cualquier paralelismo real. El tiempo registrado de 25.0464 s representa el límite de rendimiento monohilo de la CPU virtual asignada.

Al distribuir la carga en dos procesos independientes, cada instancia del intérprete opera con su propio GIL privado y el planificador CFS los asigna a núcleos físicos distintos, logrando paralelismo real a nivel de hardware. El speedup experimental de 1.83× se aproxima al teórico de 2.0×, con la brecha explicada por el overhead de fork(), la latencia de Process.join() y la contención de caché L3. Formalmente: $S = T_1/T_2 = 25.0464/13.7032 \approx 1.828$, coherente con la Ley de Amdahl para $P \approx 0.95$ y $N=2$: $S = 1/((1-P) + P/N) \approx 1.90$, indicando que aproximadamente el 5% de la carga permanece secuencial por costes de inicialización y sincronización.

Respecto al interbloqueo, su impacto en producción es crítico: un servidor web o motor de base de datos con locks mal diseñados puede quedar suspendido indefinidamente sin generar errores visibles, provocando pérdida de disponibilidad y fallos en cascada. La prevención exige imposición de un orden global en la adquisición de locks, uso de acquire(timeout=N) con manejo de excepción, y herramientas de análisis dinámico como py-spy para detectar hilos bloqueados.

En cuanto al modelo de memoria, los procesos operan sobre espacios de direcciones aislados por la MMU, garantizando fault isolation total. Los hilos comparten el heap y variables globales del proceso padre, reduciendo el coste de comunicación pero introduciendo riesgo de corrupción de estado sin sincronización. El módulo multiprocessing requiere IPC explícito mediante Queue o Pipe, mientras threading gestiona la sincronización internamente con Lock, Semaphore o Condition sobre primitivas del kernel como pthread_mutex_t. La conclusión es directa: para cargas CPU-bound, multiprocessing es la estrategia correcta; para cargas I/O-bound, threading resulta suficiente y más eficiente en consumo de memoria.

7. CONCLUSIONES

1. El módulo multiprocessing de Python 3 elude efectivamente el GIL de CPython al crear instancias independientes del intérprete mediante fork(), distribuyendo carga real en múltiples núcleos físicos. El speedup experimental de 1.83× (25.05 s → 13.70 s) para 10^9 iteraciones es coherente con la Ley de Amdahl para fracción paralelizable ~95% con dos núcleos.

2. Los interbloqueos representan un riesgo operacional crítico en sistemas concurrentes. La práctica demostró empíricamente que locks cruzados entre dos hilos satisfacen las cuatro condiciones de Coffman, produciendo congelación indefinida del programa. En producción, la prevención requiere ordenamiento de locks, timeouts en acquire() y diseño cuidadoso de jerarquías de recursos.

3. El entorno virtualizado CentOS 7 con Python 3.6.8 proporcionó un contexto experimental reproducible. La práctica evidenció que threading es adecuado para tareas I/O-bound y multiprocessing es la alternativa correcta para paralelizar cómputo intensivo. El diagnóstico correcto del tipo de carga (CPU-bound vs I/O-bound) es fundamental para el diseño de sistemas de alto rendimiento.

8. BIBLIOGRAFÍA

Coffman, E. G., Elphick, M., & Shoshani, A. (1971). System deadlocks. ACM Computing Surveys, 3(2), 67–78. https://doi.org/10.1145/356586.356588

Python Software Foundation. (2024). multiprocessing — Process-based parallelism. Python 3 Documentation. https://docs.python.org/3/library/multiprocessing.html

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating system concepts (10th ed.). John Wiley & Sons.

Tanenbaum, A. S., & Bos, H. (2015). Modern operating systems (4th ed.). Pearson Education.